

Lesson 1 — Speaker Notes

Cell-by-cell cue cards (a safety net if you get stuck) — Python from Zero for Materials Science

How to use: each entry matches one notebook cell, in order, and its first bullet is the key concept written as a full sentence you can say as-is. "▶ **RUN**" marks a code cell to execute (run it and point at the output — students are not expected to read the code). Keep the overall thread: this lecture shows WHAT Python can do; it is not a coding tutorial.

Cell 0 • TEXT • [Title / opening](#)

- **Key concept:** this lecture is a guided tour of WHAT Python can do for materials science — it is a demonstration, not a coding exercise, so nobody is expected to write or memorise code today.
- Say the guiding workflow out loud: from raw data we read the file, inspect and clean it, calculate the quantities we care about, plot them, save the results, and finally interpret them.
- Reassure the room: the aim of these two hours is to build a mental map; the actual skills come later in the hands-on sessions.

Cell 1 • TEXT • [0. Roadmap](#)

- **Key concept:** by the end of the lecture everyone should have a clear picture of why Python is useful and what a scientific analysis with it looks like, even without understanding every line.
- Walk quickly through the six goals: why Python, what the workflow looks like, what Jupyter notebooks are, the basic building blocks, why we use NumPy/pandas/matplotlib, and what students will be able to do by the end of the course.
- Repeat the reassurance: this first lecture is a map, and the hands-on sessions are where the skills are actually built.

Cell 2 • TEXT • [1. What is Python?](#)

- **Key concept:** Python is a free, general-purpose language in which you write readable instructions and the computer carries them out — for us, it behaves like a lab notebook that can also compute.
- Make the image concrete: instead of clicking through a spreadsheet, you DESCRIBE each step (read the data, compute stress and strain, plot, save), and that description itself becomes an exact, repeatable, shareable record of what you did.
- Mention the four properties that matter to us: it is readable, it is interactive (you see each result immediately), it comes with a huge ecosystem of scientific libraries, and it is free and runs everywhere — even inside a web browser.

- **Punchline:** Python is not a goal in itself; it is a precise and repeatable way to turn raw measurements into scientific results.

Cell 3 • TEXT • 2. Why Python

- **Key concept:** scientific data arrives from many sources, and Python lets us handle all of them in one consistent, repeatable way instead of by hand.
- Describe the common manual workflow — instrument output, into a spreadsheet, manual calculations, a figure, a report — and note that it works for small jobs but becomes fragile when the dataset grows, when the analysis must be repeated, or when someone else has to understand it.
- State the benefit clearly: Python makes the analysis more automatic, transparent, repeatable and shareable.
- Frame the motivation: we are not learning Python to 'learn programming', we are learning it because it lets us analyse scientific data better.

Cell 4 • TEXT • 3. Jupyter Notebook

- **Key concept:** a Jupyter notebook is a single document that mixes code, explanatory text, equations, tables, plots and their outputs together.
- Explain the mechanics: the notebook is divided into cells, you run them one at a time, and the ORDER in which you run them matters.
- Set the standard: a good notebook should not just be a place to try code — it should become a reproducible story of the analysis that someone else can follow.

Cell 5 • TEXT • Markdown cell example

- **Key concept:** a markdown cell holds formatted text and is NOT executed as Python — it is how you explain and document the analysis around your code.
- Point out that you can add titles, descriptions and notes so the notebook reads like a document, not just code.

Cell 6 • RUN • print example

- **Key concept:** a code cell contains Python that runs when you execute it, and its output appears right below it.
- **RUN it:** this is the simplest possible example — it just prints one line of text.

Cell 7 • TEXT • 4. The tensile-test example

- **Key concept:** the whole course follows one running example — the tensile test, in which a sample is pulled until it deforms and eventually breaks while the machine records the applied force and the elongation.
- Make the central point: the raw data the machine gives us (force and displacement) are not the quantities we actually want to discuss scientifically (stress and strain), so we will have to compute them ourselves.

Cell 8 • TEXT • [A picture of the experiment](#)

- **Key concept:** before any numbers, the experiment in one picture is — we pull a sample of known initial diameter and gauge length, record the force and the elongation, and from those we obtain the stress-strain curve, which is the scientific summary of the test.
- Tell them the next cell draws exactly this picture.

Cell 9 • RUN • [Schematic figure \(matplotlib\)](#)

- **Key concept:** this single figure summarises the whole experiment — the specimen being pulled on the left, and the stress-strain response it produces on the right.
- **RUN it** and point at the picture rather than the code: on the left the dog-bone specimen pulled by the force F , with its diameter and gauge length marked.
- On the right, point out the three landmarks of the curve: the elastic region (its slope is the stiffness), the peak or UTS (the strength), and the fracture point.
- Closing remark: this entire figure was drawn by matplotlib — the very tool we will use later on the real data.

Cell 10 • TEXT • [The file to look at](#)

- **Key concept:** everything in this section refers to one file, `tensile_raw.csv`, which is already sitting in the `data/` folder next to the notebook — there is nothing to download or upload.
- Invite them to open it from the file browser on the left to follow along.

Cell 11 • TEXT • [Raw columns & calculated quantities](#)

- **Key concept:** the file stores the RAW measured columns, and from them we will CALCULATE the scientific quantities we actually care about.
- Walk the raw columns: `sample_id` (which specimen), `treatment` (the material condition), `diameter` and `gauge length` (the sample geometry), and `force` and `displacement` (what the machine records).
- Then walk the calculated ones: `area` from the `diameter`, `stress` = force/area , `strain` = $\text{displacement}/\text{gauge length}$, the maximum stress as a strength indicator, and the approximate elastic modulus as the initial slope.
- Tie it together: this is exactly the raw $\rightarrow$ calculated $\rightarrow$ plot $\rightarrow$ interpret arc.

Cell 12 • TEXT • [5. The data files](#)

- **Key concept:** all the data you need is already bundled with the notebooks, and a CSV file is simply a plain-text table — a header line with the column names, then one observation per line, with values separated by commas.
- Today we mainly use `tensile_raw.csv`, which is 600 rows by 6 columns.
- Introduce the distinction we will keep all course long: raw data are usually LONG (many rows per sample), and much of the analysis is turning them into a SHORT, meaningful summary (one row per sample).

Cell 13 • TEXT • Example of a raw CSV

- **Key concept:** looking at the real first lines of the file, the same sample (S01) appears on many rows because a single tensile test records hundreds of force-displacement points for the same specimen.
- Use this to make the 'long shape' of instrument data concrete.

Cell 14 • TEXT • 6. Basic Python concepts

- **Key concept:** with the example in mind, we now meet the six basic building blocks of Python — variables, data types, lists, if statements, for loops, and functions.
- Frame them as six small, simple ideas that we will combine at the end.

Cell 15 • TEXT • 6.1 Variables

- **Key concept:** a variable is simply a NAME attached to a value, created with the '=' sign.
- Stress the crucial subtlety: '=' means ASSIGN, not 'equals' — `stress_MPa = 350` is a command read right-to-left, 'take the value 350 and store it under this name', not a mathematical statement.
- Illustrate with `n = n + 1`, which is perfectly normal in Python even though it is nonsense as an equation, and mention that the real equality TEST is the double equals '=='.
- Add the good-practice note: scientific variable names should carry meaning and unit, like `force_N` or `diameter_mm`.

Cell 16 • RUN • Variables demo

- **Key concept:** here we store two measured values in named variables and compute a third one from them.
- **RUN it:** `force_N = 1200` and `area_mm2 = 19.63` give `stress ≈ 61`; point at the comment reminding that '=' is assignment, not equality.

Cell 17 • TEXT • Discussion point

- **Key concept:** readable variable names are not about clever code — they reduce ambiguity, which is exactly what matters in science.
- Ask the room which version is clearer: the one with `x`, `A`, `y`, or the one with `force_N`, `area_mm2`, `stress_MPa`.

Cell 18 • TEXT • 6.2 Data types

- **Key concept:** values in Python have different TYPES, and the type determines what you can do with the value.
- Give the everyday cases: numbers are for calculations, text (strings) for names and labels, booleans (True/False) for decisions, and lists for collections of values.

Cell 19 • RUN • Data types demo

- **Key concept:** the `type()` function reports the type of any value.
- **RUN it** and point at the output: a string, a float, a boolean and a list.

Cell 20 • TEXT • 6.3 Lists

- **Key concept:** a list stores several values under one name — for example a set of sample names, temperatures, or hardness measurements.
- Frame it as 'a row of values you can carry around together'.

Cell 21 • RUN • List demo

- **Key concept:** from a list you can pick an item by position and compute things over the whole collection.
- **RUN it** and show: indexing with [0] gives the first value, len() gives the count, and sum()/len() gives the mean.

Cell 22 • TEXT • Indexing note

- **Key concept:** in Python counting starts at ZERO, so the first item sits at position 0, not 1 — this is the single most common beginner surprise.
- Mention that later we will use NumPy arrays and pandas columns, which are more powerful than plain lists.

Cell 23 • TEXT • 6.4 If statements

- **Key concept:** an if statement lets the program make a DECISION — do one thing when a condition holds, and something else otherwise.
- Give materials examples: above a threshold, belonging to a treatment group, including or excluding a measurement, elastic versus plastic region.

Cell 24 • RUN • If demo

- **Key concept:** the condition is checked and the matching branch runs.
- **RUN it:** stress = 450 is greater than 400, so it prints 'High strength'; you can change the number live and re-run.

Cell 25 • TEXT • If notes

- **Key concept:** this is the same reasoning we use in science — if a value satisfies a criterion we act one way, otherwise another.
- Add the good practice: a threshold in code should always have a scientific reason behind it.

Cell 26 • TEXT • 6.5 For loops

- **Key concept:** a for loop repeats the SAME operation many times — over several samples, files, temperatures, or conditions.
- This is the heart of automation: doing once what you would otherwise do by hand many times.

Cell 27 • RUN • For loop: samples

- **Key concept:** the loop runs its body once for each item in the list.
- **RUN it:** it walks through three sample names and prints a line for each.

Cell 28 • RUN • For loop: temperatures

- **Key concept:** the same operation can be applied to every value in a list automatically.
- **RUN it:** it converts a list of temperatures from Celsius to Kelvin, one after another.

Cell 29 • TEXT • For notes

- **Key concept:** whenever you catch yourself repeating the same action by hand, that action can probably be automated with a loop.
- Mention that in numerical Python we often replace explicit loops with vectorised NumPy/pandas operations.

Cell 30 • TEXT • 6.6 Functions

- **Key concept:** a function is a named, reusable calculation — a recipe that takes inputs, does something, and returns an output.
- Point out how natural this is in science, where so many calculations are just formulas.

Cell 31 • RUN • Function: stress

- **Key concept:** you define the calculation once and then reuse it whenever you need it.
- **RUN it:** `compute_stress(1200, 19.63)` returns the stress, computed by the function we just defined.

Cell 32 • RUN • More functions

- **Key concept:** several small functions can each capture one formula clearly.
- **RUN it:** Celsius-to-Kelvin, strain, and circular area; point at `math.pi` as a value taken from the `math` library using dot-notation.

Cell 33 • TEXT • Functions note

- **Key concept:** functions keep notebooks clean, avoid repetition, and separate the scientific logic from the technical details.

Cell 34 • TEXT • 7. Libraries (what is a library?)

- **Key concept:** a library is a collection of ready-made, already-tested code that someone else wrote and shared, which you `IMPORT` to instantly gain a set of new tools.
- Use the lab analogy: just as you don't build your own balance, furnace or diffractometer, you don't re-write how to read a file or draw a curve — you rely on a trusted library and focus on your science.
- Explain the import line as 'taking a library off the shelf and giving it a short nickname', e.g. `import numpy as np`.
- Name the three we use: NumPy for numbers and arrays, pandas for tables, and matplotlib for plots.

Cell 35 • TEXT • 7.1 NumPy

- **Key concept:** NumPy's core object is the array, which lets us do arithmetic on many numbers all at once instead of one at a time.

- Make it concrete: rather than writing a loop, you write `stress = force / area` once and it is applied to every value together — shorter to read and much faster.

Cell 36 • RUN • NumPy demo

- **Key concept:** a single expression operates on the whole array.
- **RUN it:** dividing a force array by the area yields a whole array of stresses in one line.

Cell 37 • TEXT • NumPy note

- **Key concept:** the ability to calculate many numbers at once is why NumPy arrays are the foundation of the entire scientific Python ecosystem.

Cell 38 • TEXT • 7.2 pandas

- **Key concept:** pandas is for working with TABLES, and its main object — the DataFrame — is like a spreadsheet, except every operation is written explicitly so it is reproducible.
- Note that each column behaves like a named NumPy array, so the column arithmetic we just saw works directly.
- Frame its role: pandas is where most everyday work happens — it is the bridge from a raw CSV file to a clean table of results.

Cell 39 • RUN • Load the data

- **Key concept:** this cell reads `tensile_raw.csv` into a DataFrame called `df` that later cells reuse.
- **RUN it;** treat the loader function as plumbing — say 'normally this is just `pd.read_csv(...)`' — and point at the printed shape (600, 6).

Cell 40 • RUN • `df.head()`

- **Key concept:** `.head()` shows the first rows so you can eyeball the table.
- **RUN it:** the same data they saw as raw text now appears as a proper table.

Cell 41 • RUN • Shape & columns

- **Key concept:** `.shape` and `.columns` tell you the size and the column names of the table.
- **RUN it** and read out the shape and the list of columns.

Cell 42 • TEXT • DataFrame note

- **Key concept:** a DataFrame is like a spreadsheet, but because the operations are written explicitly they are reproducible and easy to repeat on new data.

Cell 43 • TEXT • 7.3 matplotlib

- **Key concept:** matplotlib is the tool for scientific plots — in fact it already drew the tensile-test schematic earlier in this notebook.
- Describe the usual pattern: open a figure, draw the data, then add the labels, units and title that make it scientific.
- Stress that a figure is part of the RESULT, not decoration: a curve with no labels or units cannot be interpreted by anyone else.

Cell 44 • RUN • Plot demo

- **Key concept:** a proper plot always carries axis labels, units and a title.
- **RUN it** and point at each element of the small stress-strain example: the line, the labelled axes, the title, the grid.

Cell 45 • TEXT • [matplotlib note](#)

- **Key concept:** plotting is not only a technical step — figure quality is part of scientific communication.

Cell 46 • TEXT • [8. Mini demo \(intro\)](#)

- **Key concept:** now we combine all the pieces into one short, complete workflow — import the tools, read the data, calculate new columns, select what we want, and plot it.
- Remind them (gently, once) that the goal is to watch the whole arc, not to understand each line.

Cell 47 • RUN • [The full pipeline](#)

- **Key concept:** this is the entire analysis on real data, from file to figure, in one cell.
- **RUN it** and follow the four numbered comments — read, calculate area/stress/strain, select sample S01, plot its curve — then point at the resulting S01 stress-strain curve.

Cell 48 • TEXT • [What happened in the demo](#)

- **Key concept:** in five steps we imported tools, read a file, calculated new columns, selected one sample and made a plot — that is a full scientific mini-workflow.
- **Looking ahead:** in the hands-on sessions students will learn to write and modify each of these parts themselves.

Cell 49 • TEXT • [Same workflow, all samples](#)

- **Key concept:** with almost the same code, the analysis can be repeated for every sample at once, so one curve becomes the whole set.

Cell 50 • RUN • [All curves \(groupby\)](#)

- **Key concept:** grouping by sample and looping lets us draw every specimen on one chart.
- **RUN it** and point out that the tougher samples (e.g. the quenched one) sit higher on the plot.

Cell 51 • TEXT • [Note](#)

- **Key concept:** this is where loops and grouping pay off — the same analysis is repeated for many samples without copying and pasting code.

Cell 52 • TEXT • [Numerical summary by treatment](#)

- **Key concept:** the raw curves can also be turned into a compact table of results grouped by heat treatment — a first glimpse of where the course is heading.

Cell 53 • RUN • Summary per sample

- **Key concept:** grouping reduces the long table to one row per sample with its key numbers.
- **RUN it:** it produces the maximum stress and maximum strain for each sample.

Cell 54 • RUN • Summary per treatment

- **Key concept:** grouping one level further gives a genuine comparison between material conditions.
- **RUN it:** it shows the mean and standard deviation of the maximum stress for each treatment.

Cell 55 • TEXT • Note

- **Key concept:** real research output is not only a plot but also numerical summaries that can be saved, checked and reported.

Cell 56 • RUN • Save results (to_csv)

- **Key concept:** to_csv writes our results back out to a file — this is the 'save results' step of the workflow.
- **RUN it:** a new CSV appears in the file browser on the left.

Cell 57 • TEXT • 9. Errors & debugging

- **Key concept:** errors are completely normal and do not mean you are bad at programming — they simply mean Python found something inconsistent.
- Read through the common beginner errors: a misspelled name, a missing parenthesis, a file not found, a cell not executed, a wrong column name, the wrong run order, or the wrong CSV separator.

Cell 58 • RUN • Error demo (safe)

- **Key concept:** this cell is safe to run because the error-producing line is commented out.
- **RUN it** as-is; optionally, uncomment print(forcee) to trigger a real NameError live, then comment it back.

Cell 59 • TEXT • Explain the error

- **Key concept:** if you uncomment the line, Python raises a NameError because 'force' and 'forcee' are two different names — a simple typo.
- Reassure them: the error message looks intimidating but is useful, because it tells you where Python got stuck and often why.

Cell 60 • TEXT • 10. Bonus: animations (intro)

- **Key concept:** with just a few extra lines, a static plot can become an animation with a play button and a draggable slider, which is a nice way to FEEL how the numbers behave.
- **Practical note:** these use only matplotlib and take a couple of seconds to generate — run the cell, wait for the player, then press play or drag the slider.

- Set expectations: the point here is not the code, it is to show that the same read → compute → plot logic can become interactive.

Cell 61 • RUN • Bonus setup

- **Key concept:** this cell reloads the data and recomputes the columns so the following animations are self-contained.
- **RUN it:** it just prints 'Ready'.

Cell 62 • TEXT • 10.1 Diameter sweep (intro)

- **Key concept:** the machine measures force, but engineers care about stress = force/area, and since the area depends on the diameter, a thinner specimen carries the same force over a smaller area and therefore reaches a higher stress.
- The animation sweeps S01's diameter from thick to thin to show this.

Cell 63 • RUN • 10.1 animation

- **Key concept:** as the diameter shrinks, the peak stress rises even though the force is unchanged.
- **RUN it,** then press play or drag the slider to control the diameter yourself.

Cell 64 • TEXT • 10.2 Walk the test (intro)

- **Key concept:** this animation walks through the test point by point, so you can watch the force, stress and strain evolve from the first gentle pull all the way to fracture.

Cell 65 • RUN • 10.2 animation

- **Key concept:** the moving dot traces the specimen's state along the stress-strain curve.
- **RUN it,** then play or scrub with the slider.

Cell 66 • TEXT • Mini-gallery (intro)

- **Key concept:** the next animations do not use the tensile data at all — they simply show how a few lines of Python can make a physical idea move, and each cell is self-contained.

Cell 67 • TEXT • 10.3 Atoms melting

- **Key concept:** as the temperature rises, each atom in the lattice takes bigger random steps and the orderly crystal gradually falls apart — that is melting.

Cell 68 • RUN • 10.3 animation

- **RUN it,** then press play.

Cell 69 • TEXT • 10.4 Diffusion

- **Key concept:** many particles start together at the centre and take random steps, spreading out over time — this is how heat, atoms or ink diffuse through a material.

Cell 70 • RUN • 10.4 animation

- **RUN it,** then press play.

Cell 71 • TEXT • [10.5 XRD](#)

- **Key concept:** as the detector sweeps the angle 2θ , sharp peaks appear at specific angles, and that pattern is the fingerprint used to identify a crystal structure.

Cell 72 • RUN • [10.5 animation](#)

- RUN it, then press play.

Cell 73 • TEXT • [10.6 Atoms on springs](#)

- **Key concept:** a chain of atoms connected by springs oscillating in one of its natural modes is the microscopic picture behind elasticity.

Cell 74 • RUN • [10.6 animation](#)

- RUN it, then press play.

Cell 75 • TEXT • [10.7 Travelling wave](#)

- **Key concept:** a wave packet moving through the material is like an ultrasonic pulse used to inspect a part for hidden defects.

Cell 76 • RUN • [10.7 animation](#)

- RUN it, then press play.

Cell 77 • TEXT • [10.8 Fourier square wave](#)

- **Key concept:** adding more and more smooth sine waves together can build up a sharp square wave — the basic idea of Fourier analysis.

Cell 78 • RUN • [10.8 animation](#)

- RUN it, then play and watch the shape sharpen as terms are added.

Cell 79 • TEXT • [10.9 Monte Carlo \$\pi\$](#)

- **Key concept:** by throwing random points into a square and counting the fraction that land inside the circle, we can estimate the value of π .

Cell 80 • RUN • [10.9 animation](#)

- RUN it, then play and watch the estimate converge.

Cell 81 • TEXT • [10.10 Double pendulum](#)

- **Key concept:** the double pendulum is chaotic — a tiny change in the starting conditions leads to a completely different path, so it looks different every run.

Cell 82 • RUN • [10.10 animation](#)

- RUN it, then press play.

Cell 83 • TEXT • [10.11 Heat plate](#)

- **Key concept:** a plate with a few hot spots gradually evens out as the heat diffuses, exactly as heat conduction works in a real material.

Cell 84 • RUN • 10.11 animation

- **RUN it**, then press play.

Cell 85 • TEXT • 10.12 Free-energy surface (3D)

- **Key concept:** on a free-energy surface the height is the energy and the valleys are the stable states of the material, separated by an energy barrier — and this one you can rotate freely with the mouse.
- **Heads-up:** the first run installs the plotly library in the browser, so it takes a few extra seconds.

Cell 86 • RUN • 10.12 plotly

- **Key concept:** this is a genuinely interactive 3D plot.
- **RUN it**, wait for the plotly install, then drag to rotate and scroll to zoom.

Cell 87 • TEXT • 11. Closing

- **Key concept:** by the end of the course students should be able to read a data file, inspect a table, calculate new quantities, filter and group data, make scientific plots, save results, and write a clear reproducible notebook.
- Be explicit about what we will NOT cover in depth: advanced software development, object-oriented programming, machine learning, advanced simulations, full XRD analysis, advanced image analysis, and packaging.
- **Final message:** the goal is not to become professional software developers, but to become autonomous in everyday scientific data analysis.

Cell 88 • TEXT • References

- **Key concept:** the final cell collects the official documentation for everything shown, for anyone who wants to read more on their own.
- Just point to it — there is no need to read the links aloud.